arXiv:2401.08671v1 [cs.PF] 9 Jan 2024

DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference

Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, Yuxiong He

Microsoft DeepSpeed (www.deepspeed.ai)

Abstract

The deployment and scaling of large language models (LLMs) have become critical as they permeate various applications, demanding high-throughput and low-latency serving systems. Existing frameworks struggle to balance these requirements, especially for workloads with long prompts. This paper introduces DeepSpeed-FastGen, a system that employs Dynamic SplitFuse, a novel prompt and generation composition strategy, to deliver up to 2.3x higher effective throughput, 2x lower latency on average, and up to 3.7x lower (token-level) tail latency, compared to state-of-the-art systems like vLLM. We leverage a synergistic combination of DeepSpeed-MII and DeepSpeed-Inference to provide an efficient and easy-to-use serving system for LLMs. DeepSpeed-FastGen's advanced implementation supports a range of models and offers both non-persistent and persistent deployment options, catering to diverse user scenarios from interactive sessions to long-running applications. We present a detailed benchmarking methodology, analyze the performance through latency-throughput curves, and investigate scalability via load balancing. Our evaluations demonstrate substantial improvements in throughput and latency across various models and hardware configurations. We discuss our roadmap for future enhancements, including broader model support and new hardware backends. The DeepSpeed-FastGen code is readily available for community engagement and contribution.

1 Introduction

Large language models (LLMs) like GPT-4 [1] and LLaMA [2] have emerged as a dominant workload in serving a wide range of applications infused with AI at every level. From general chat models to document summarization, and from autonomous driving to copilots at every layer of the software stack, the demand to deploy and serve these models at scale has skyrocketed. While frameworks like DeepSpeed, PyTorch [3], and several others can regularly achieve good hardware utilization during LLM training, the interactive nature of these applications and the poor arithmetic intensity of tasks like open-ended text generation have become the bottleneck for inference throughput in existing systems.

To this end, frameworks like vLLM [4] powered by PagedAttention and research systems like Orca [5] have significantly improved the performance of inference for LLMs. However, these systems still struggle to provide consistent quality of service, particularly for workloads with longer prompts. These long prompt workloads are becoming increasingly important as more and more models, like MPT-StoryWriter [6], and systems, such as DeepSpeed Ulysses [7],

support context windows stretching to tens of thousands of tokens. To better understand the problem space, we provide detailed examples of how text generation works for LLMs in two distinct phases called prompt processing and generation. When systems treat them as distinct phases, generation will be preempted by prompt processing that risks breaking the service level agreements (SLAs).

Today, we are glad to present DeepSpeed-FastGen, a system that overcomes these limitations by leveraging the proposed Dynamic SplitFuse technique and offers up to 2.3x higher effective throughput, 2x lower latency on average, and up to 3.7x lower (token-level) tail latency, compared to state-of-the-art systems like vLLM. DeepSpeed-FastGen leverages the combination of DeepSpeed-MII and DeepSpeed-Inference to provide an easy-to-use serving system.

2 Existing LLM Serving Techniques in Literature

A text generation workload for a single sequence consists of two phases: 1) prompt processing, in which the user-provided text is efficiently processed as a batch of tokens to build a key-value (KV) cache for attention, and 2) token generation, which will add a single token to that cache and generate a new token. Over the course of generating a sequence of text, the model will make many forward calls to the model to generate the full sequence of text. Two major techniques have been proposed in the literature and deployed in systems that address various limitations and bottlenecks that may arise during these phases.

2.1 Blocked KV Caching

vLLM identified that memory fragmentation due to large monolithic KV-caches significantly reduced the concurrency of LLM serving systems and proposed Paged Attention [8] to enable non-contiguous caches and increase total system throughput. Rather than assign individual variable-sized contiguous chunks of memory, the underlying storage in the KV cache is fixed-sized blocks (also known as pages). The blocked KV-cache increases system throughput by increasing the amount of potential sequence concurrency by eliminating KV-cache induced memory fragmentation. Non-contiguous KV cache implementations are also included in HuggingFace TGI [9] and NVIDIA TensorRT-LLM [10].

2.2 Continuous Batching

In the past, dynamic batching, in which a server would wait for multiple requests to process in phase with each other, was used to improve GPU utilization. However, this approach has drawbacks, as it typically requires padding inputs to identical lengths or stalling the system to wait to construct a larger batch.

Recent advancement in large language model (LLM) inference and serving has been focusing on fine granularity scheduling and optimizing memory efficiency. For instance, Orca proposes iteration-level scheduling (also known as continuous batching) which makes distinct scheduling decisions at each forward pass of the model. This allows requests to join/leave the batch as needed, eliminating the need for padding requests thus improving the overall throughput. In addition to Orca, continuous batching has been implemented in NVIDIA TRT-LLM, HuggingFace TGI, and vLLM.

In current systems, there are two primary approaches to implement continuous batching. In TGI and vLLM, the generation phase is preempted to perform prompt processing (called infill in TGI) before continuing with generation. In Orca, these phases are not distinguished; instead, Orca will add a prompt into the running batch so long as the total number of sequences

doesn't reach a fixed bound. These approaches to varying degrees need to stall generation to process long prompts (see Section 3.2).

We propose a novel prompt and generation composition strategy, Dynamic SplitFuse discussed at length in the next section.

3 Dynamic SplitFuse: A Novel Prompt and Generation Composition Strategy

DeepSpeed-FastGen is built to leverage continuous batching and non-contiguous KV caches to enable increased occupancy and higher responsivity for serving LLMs in the data center, similar to existing frameworks such as TRT-LLM, TGI, and vLLM. iIn order to achieve a new level of performance, DeepSpeed-FastGen introduces SplitFuse which leverages dynamic prompt and generation decomposition and unification to further improve continuous batching and system throughput.

3.1 Three Performance Insights

Before describing Dynamic SplitFuse, we answer three key performance questions that together motivate its design.

3.1.1 What factors impact the forward pass of a single LLM?

In order to effectively schedule, it is necessary to understand what are the relevant independent variables the scheduling loop should control. We observe below that the composition of sequences in a forward pass (the batch size in sequences) has a negligible impact on performance compared to the raw number of tokens in the forward pass. This means an effective scheduler can be built around a single signal, the number of tokens in the forward pass.

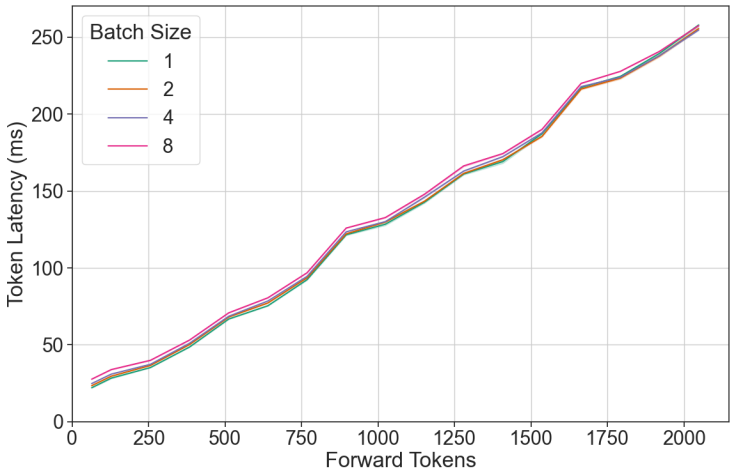

Figure 1: Token latency (ms) is predominantly determined by the number of forward tokens rather than the batch sizes.

3.1.2 How does a model's throughput respond to changing the number of tokens in the forward pass?

An LLM has two key operating regions with a relatively steep transition. With a small number of tokens, the GPU bottleneck is reading the model from memory and so throughput scales with the number of tokens, whereas with many tokens the model is throughput bound by compute and sees near-constant throughput. The model should run highly efficiently if all forward passes are in the throughput-saturating region.

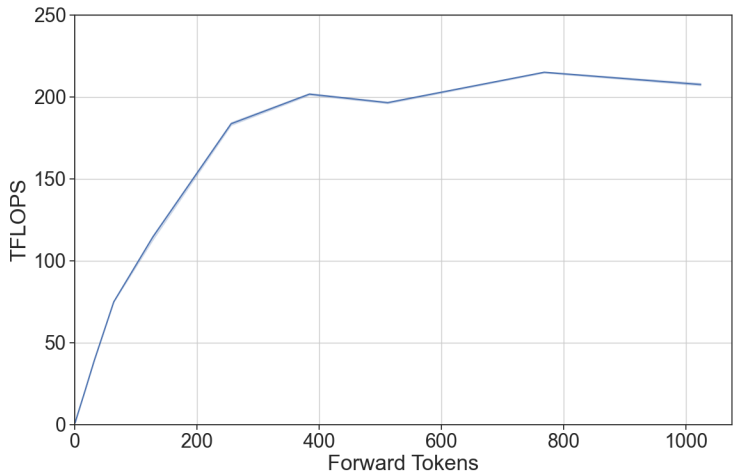

Figure 2: The system reaches peak performance (throughput saturation region) as the number of tokens in the forward pass increase. Beyond that, near-constant throughput is observed.

3.1.3 How should a pool of tokens be scheduled across multiple forward passes?

We observe above that for well-aligned inputs the token-throughput curve is concave, which means the second derivative is bound to be less than or equal to 0. As an example, let $f(x)$ be a concave function of latency to throughput for a given model. For a concave function $f(x)$, the following holds:

$$0 \geq \lim_{h \to 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

$$0 \geq f(x+h) - 2f(x) + f(x-h)$$

$$2f(x) \geq f(x+h) + f(x-h)$$

This states that for a given pool of 2x tokens to process, the manner that maximizes throughput is that which evenly splits them between two batches. More generally, in a system that must consume and process P tokens over F forward passes, the ideal partitioning scheme will divide them equally.

3.2 Dynamic SplitFuse

Dynamic SplitFuse is a novel token composition strategy for prompt processing and token generation. DeepSpeed-FastGen utilizes Dynamic SplitFuse to run at a consistent forward size by leveraging the capability to take partial tokens from prompts and compose this with generation. A similar approach has been proposed in Sarathi[11] where it splits a prompt into smaller chunks to combine more token generation with prompt processing and to run forward passes with consistent batch sizes. In particular, Dynamic SplitFuse performs two key behaviors:

1. Long prompts are decomposed into much smaller chunks and scheduled across multiple forward passes (iterations) with only the final pass performing any generation.
2. Short prompts will be composed to exactly fill a target token budget. Even short prompts may be decomposed to ensure the budget is precisely met and the forward sizes are well-aligned.

Together, these two techniques provide concrete benefits on all user metrics:

1. Better Responsiveness: Since long prompts no longer require extremely long forward passes to process, the model will provide lower client latency. More forward passes are performed within the same window of time.
2. Higher Efficiency: Fusion of short prompts to larger token budgets enables the model to consistently operate in the high throughput regime.
3. Lower variance and better consistency: Since forward passes are of consistent size and forward pass size is the primary determinant of performance, the latency of each forward pass is much more consistent than competing systems as is the perceived generation frequency. There are no preemption or long-running prompts to increase the latency as in other prior work. This translates to a reduction of up to 3.7x P95 latency in generation as we show in Section 4.

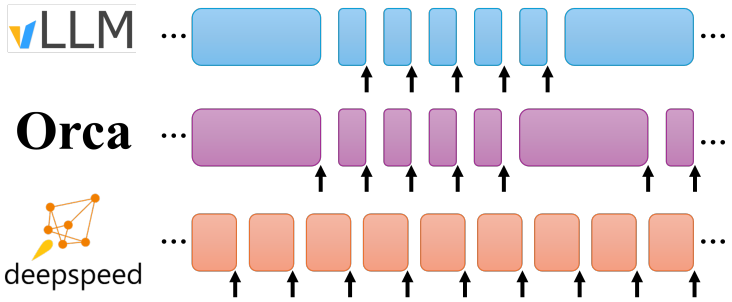

Figure 3: Illustration of continuous batching strategies. Each block shows the execution of a forward pass. An arrow indicates that the forward pass has sequences with one or more tokens generated. vLLM performs either token generations or prompt processing in a forward pass; token generation preempts prompt processing. Orca runs prompts at their complete length alongside generation. Dynamic SplitFuse performs dynamic composition of fixed-sized batches composed of both generation and prompt tokens.

Consequently, DeepSpeed-FastGen will consume tokens from incoming prompts at a rate that permits fast ongoing generation while adding tokens to the system that increase system utilization, providing lower latency and higher throughput streaming generation to all clients as compared to other state-of-the-art serving systems.

4 Performance Evaluation

DeepSpeed-FastGen provides state-of-the-art LLM serving performance leveraging its blocked KV cache and Dynamic SplitFuse continuous batching. We evaluate DeepSpeed-FastGen against vLLM [4] on a range of models and hardware configurations following the benchmarking methodology discussed below. The evaluation shows that DeepSpeed-FastGen achieves up to 2.3x higher effective throughput, 2x lower latency on average, and up to 3.7x lower (token-level) tail latency, compared to state-of-the-art systems like vLLM.

4.1 Benchmarking Methodology

We use two primary quantitative schemes for measuring performance.

4.1.1 Throughput-Latency Curves

Two key metrics for production readiness are throughput (measured in requests per second) and latency (the responsiveness of each request). To measure this, we instantiate multiple clients (ranging from 1 to 32) concurrently and send requests (512 in total) to the server. The resulting latency of each request is measured at the endpoint and throughput is measured by the end-to-end time to complete the experiment.

4.1.2 Effective Throughput

Interactive applications, such as chat applications, can have more stringent and complex requirements than can be captured by top-level metrics like end-to-end latency. In particular, we focus on the increasingly popular chat user scenario:

1. A user initiates a task by sending a prompt.
2. The system processes the prompt and returns the first token.
3. Subsequent tokens are streamed to the user as they are produced.

At each point in this process there is an opportunity for a system to provide an adverse user experience; for example, if the first token arrives too slowly or the generation appears to stop for some time. We propose an SLA framework that considers both of these dimensions.

As the lengths of prompts and generated texts vary significantly, affecting computational costs, it is impractical to set rigid SLA values for throughput and latency. Therefore, we define the SLA for prompt latency as $\frac{\text{—tokens in prompt—}}{512}$ seconds (=512 tokens/s). Additionally, considering humans' reading speed, we set the SLA for generation latency on the Exponential Moving Average (EMA) to 2, 4, or 6 tokens/sec. Requests that adhere to these SLAs are deemed successful, and the throughput of these successful requests is referred to as **effective throughput**.

We evaluate vLLM and DeepSpeed-FastGen on both Llama-2 7B, Llama-2 13B, and Llama-2 70B [2] on NVIDIA A100, H100, and A6000.

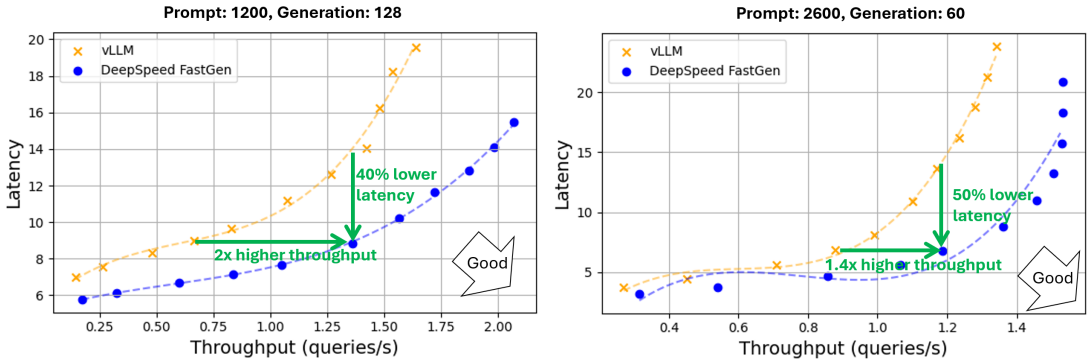

Figure 4: Throughput and latency of text generation using Llama 2 70B (Tensor parallelism across 4 A100-80GB GPUs). A normal distribution was applied to prompt and generation lengths with averages of 1200/2600 and 128/60, respectively, and a 30% variance.

4.2 Throughput-Latency Analysis

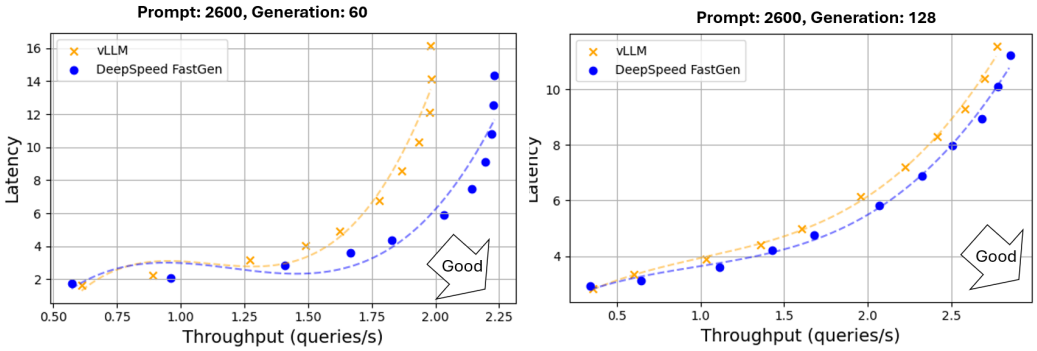

Figure 5: Throughput and latency of text generation using Llama 2 13B (A100-80GB GPU, no tensor parallelism). A normal distribution was applied to prompt and generation lengths with averages of 1200/2600 and 60/128, respectively, and a 30% variance.

In this experiment, DeepSpeed-FastGen outperforms vLLM in both throughput and latency, providing equivalent latency with greater throughput or more responsive latency and the same throughput. On Llama-2 70B with 4 A100x80GB, DeepSpeed-FastGen demonstrates up to 2x higher throughput (1.36 rps vs. 0.67 rps) at identical latency (9 seconds) or up to 50% latency reduction (7 seconds vs. 14 seconds) while achieving the same throughput (1.2 rps), as shown in Figure 2. These trends hold when evaluating Llama-2 13B as shown in Figure 3.

4.3 Effective Throughput Analysis

Under the effective throughput analysis that considers both first token latency and the rate at which generation occurs, DeepSpeed-FastGen provides up to 2.3x higher throughput than vLLM. Figure 4 presents a comparative analysis of the effective throughputs of DeepSpeed-FastGen and vLLM. Each plotted point denotes the effective throughput derived from a spe-

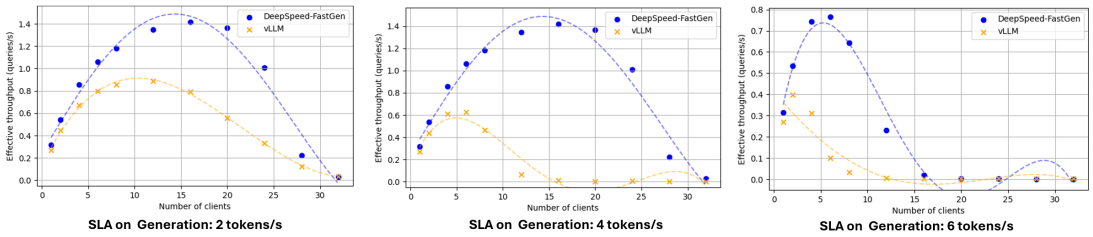

Figure 6: Effective throughput of DeepSpeed-FastGen and vLLM (Llama 2 70B/A100-80GB using tensor parallelism across 4 A100-80GB GPUs. A normal distribution was applied to prompt and generation lengths with averages of 2600 and 60, respectively, and a 30% variance).

cific number of clients. As we scaled the number of clients, we initially observed an increase in effective throughput. However, the latency also significantly increases as the number of clients approaches the system's capacity, causing many requests to fail in meeting the SLA. Consequently, the effective throughput will either saturate or decrease at some point. From a usability perspective, it's not particularly relevant how many clients are required to achieve the max effective throughput; the maximum point of the line is the optimal serving point.

4.4 Significant Tail Latency Reduction for Token Generation

Figure 5 displays the P50, P90, and P95 latencies of the generation processes. Both vLLM and DeepSpeed-FastGen exhibit similar P50 latencies, but vLLM demonstrates significantly higher latencies for P90 and P95. Regarding the P95 latencies, DeepSpeed-FastGen achieved a reduction of 3.7 times.

This discrepancy is due to a noticeable spike in vLLM's generation latency when it preempts the ongoing generation to process new prompts. In contrast, DeepSpeed-FastGen typically processes the prompt and generation for previous requests concurrently, leading to much more consistent generation latency.

4.5 Scalability using Load Balancing

DeepSpeed-FastGen offers replica-level load balancing that evenly distributes requests across multiple servers, allowing you to effortlessly scale up your application. Figure 6 illustrates the scalability of DeepSpeed-FastGen when employing the load balancer and up to 16 replicas. Note that we utilized 4 A100 GPUs to compute the Llama 2 70B model. In total, we employed 8 nodes to run the 16 replicas. The results demonstrate nearly perfect scalability with DeepSpeed-FastGen. Given that the throughput of a single replica is 1.46 queries/sec, the throughput with 16 replicas reaches 23.7 queries/sec, marking a linear 16x increase compared to a single replica.

4.6 Other Hardware Platforms

In addition to the deep analysis on A100, we provide additional benchmarking results for H100 and A6000. The same performance trends were observed on both A6000 and H100 as A100.

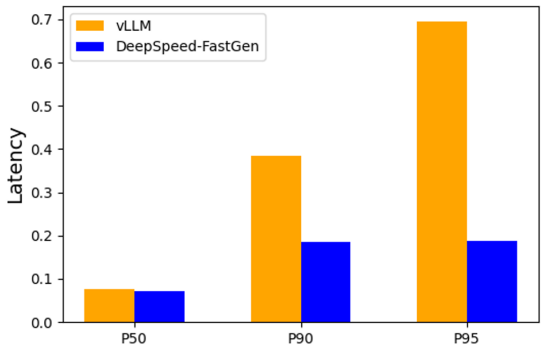

Figure 7: Per-Token generation Latency of Llama 2 70B/A100-80GB using tensor parallelism across 4 A100-80GB GPUs, 16 clients. A normal distribution was applied to prompt and generation lengths with averages of 2600 and 128, respectively, and a 30% variance.

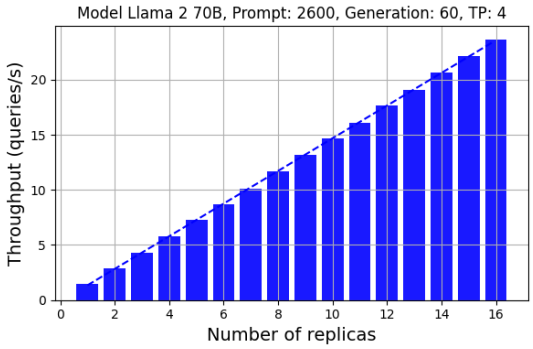

Figure 8: Scalability using the load balancing feature. A normal distribution was applied to prompt and generation lengths with averages of 2600 and 60, respectively, and a 30% variance.

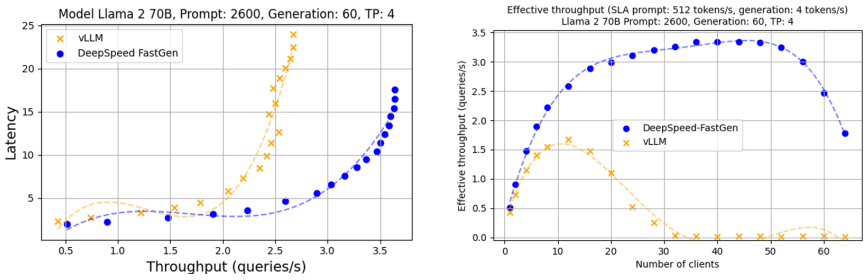

Figure 9: Throughput-latency curve and effective throughput of Llama 2 70b using 8 H100 GPUs. A normal distribution was applied to prompt and generation lengths with averages of 2600 and 60, respectively, and a 30% variance.

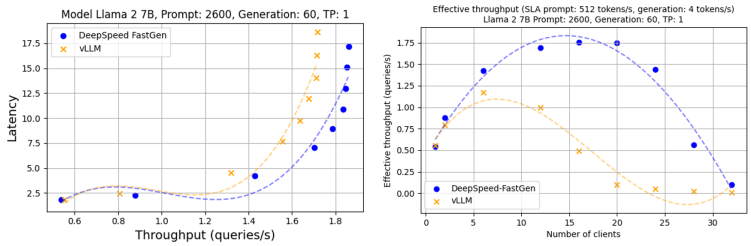

Figure 10: Throughput-latency curve and effective throughput of Llama 2 7b using A6000. A normal distribution was applied to prompt and generation lengths with averages of 2600 and 60, respectively, and a 30% variance.

5 DeepSpeed-FastGen: Implementation and Usage

DeepSpeed-FastGen is the synergistic composition of DeepSpeed-MII and DeepSpeed-Inference as illustrated in the figure below. Together, both of these software packages provide various components of the system including the frontend APIs, the host and device infrastructure to schedule batches using Dynamic SplitFuse, optimized kernel implementations, and the tools to construct new model implementations.

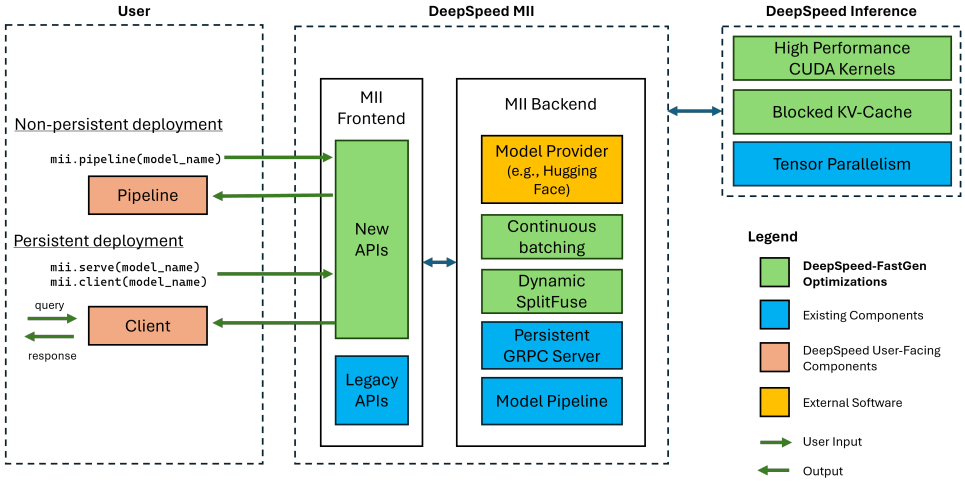

Figure 11: Architecture of DeepSpeed-FastGen

The fastest way to get started with our alpha release of DeepSpeed-FastGen is by running the following command:

```
pip install deepspeed-mii
```

5.1 Supported Models

We currently support the following HuggingFace model families[1] in this alpha release of DeepSpeed-FastGen:

- LLaMA and LLaMA-2
- Mistral
- Facebook OPT

All current models leverage HuggingFace APIs in our backend to provide both the model weights and the model's corresponding tokenizer. We plan to add additional models in the coming weeks and months after the initial release. If there are specific model architectures you would like supported, please file an issue and let us know.

5.2 Deployment Options

All of the examples below are runnable in DeepSpeedExamples. Once installed you have two options for deployment: an interactive non-persistent pipeline or a persistent serving deployment:

5.2.1 Non-persistent Pipeline

The non-persistent pipeline deployment is a great and fast way to get started and can be done with only a few lines of code. Non-persistent models are only around for the duration of the python script you are running but are useful for temporary interactive sessions.

```
from mii import pipeline
pipe = pipeline("mistralai/Mistral-7B-v0.1")
output = pipe(["Hello, my name is", "DeepSpeed is"], max_new_tokens=128)
print(output)
```

5.2.2 Persistent Deployment

A persistent deployment is ideal for use with long-running and production applications. The persistent deployment uses a lightweight GRPC server that can be created using the following 2 lines:

```
import mii
mii.serve("mistralai/Mistral-7B-v0.1")
```

The above server can be queried by multiple clients at once thanks to the built-in load balancer from DeepSpeed-MII. Creating a client also just takes 2 lines of code:

```
client = mii.client("mistralai/Mistral-7B-v0.1")
output = client.generate("Deepspeed is", max_new_tokens=128)
print(output)
```

A persistent deployment can be terminated when it is no longer needed:

```
client.terminate_server()
```

[1] https://huggingface.co/models

5.3 Advanced Installation Information

For ease of use and a significant reduction in lengthy compile times that many projects require in this space, we distribute a pre-compiled Python wheel covering the majority of our custom kernels through a new library called DeepSpeed-Kernels. We have found this library to be very portable across environments with NVIDIA GPUs with compute capabilities 8.0+ (Ampere+), CUDA 11.6+, and Ubuntu 20+. In most cases, you shouldn't even need to know this library exists as it is a dependency of DeepSpeed-MII and will be installed with it. However, if for whatever reason you need to compile our kernels manually please see our advanced installation docs.

6 Release: Try Out DeepSpeed-FastGen

We are very excited to share this DeepSpeed-FastGen alpha release. To get started, please visit our GitHub Landing Page page for DeepSpeed-MII.

DeepSpeed-FastGen is part of the bigger DeepSpeed ecosystem comprising a multitude of Deep Learning systems and modeling technologies. To learn more,

- Please visit our website for detailed blog posts, tutorials, and helpful documentation.
- You can also follow us on our English Twitter, Japanese Twitter, and Chinese Zhihu for the latest news on DeepSpeed.

DeepSpeed welcomes your contributions! We encourage you to report issues, contribute PRs, and join discussions on the DeepSpeed GitHub page. Please see our contributing guide for more details. We are open to collaborations with universities, research labs, and companies, such as those working together on deep learning research, applying DeepSpeed to empower real-world AI models and applications, and so on. For such requests (and other requests unsuitable for GitHub), please directly email to deepspeed-info@microsoft.com.

6.1 Roadmap

The following items are on our roadmap and we plan to engage with our community on these through our GitHub issues and PRs:

- Performance improvements
- Broader model support
- New hardware backends through collaboration with partners
- Release performance benchmarks (used to generate plots in this blog)

"Star" our DeepSpeed GitHub and DeepSpeedMII GitHub repositories if you like our work!

Acknowledgment

We thank the entire DeepSpeed team for their contributions on developing, debugging, testing, and releasing the DeepSpeed-FastGen software. We would like to thank various open-source community projects including HuggingFace, vLLM, and HuggingFace TGI. We have leveraged

HF APIs to support models and tokenizers in our alpha release and will continue to add more models. We especially acknowledge and thank the developers of Flash Attention[12, 13] for their great work. We have extensively leveraged FlashAttention kernels in our system with modifications that have been acknowledged in our code repositories at appropriate file headers. Finally, we want to thank the developers of FasterTransformer [14] kernels that we have used in our MoE kernels (released as part of DeepSpeed-Kernels repository).

References

[1] OpenAI. Gpt-4. https://openai.com/gpt-4, 2023.

[2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library, 2019.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[5] GI Yu, Jeong JSeong, GW Kim, S Kim, and BG Chun. Orca: A distributed serving system for {Transformer-Based} generative models. 2022.

[6] MosaicML. Introducing mpt-7b: A new standard for open-source, commercially usable llms. https://www.mosaicml.com/blog/mpt-7b, 2023.

[7] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[9] HuggingFace. Text generation inference. https://huggingface.co/text-generation-inference.

[10] NVIDIA. Nvidia tensorrt-llm: A tensorrt toolbox for large language model. https://github.com/NVIDIA/TensorRT-LLM.

[11] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[13] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. 2023.

[14] NVIDIA. Fastertransformer. https://github.com/NVIDIA/FasterTransformer.